

Started on	Wednesday, 7 October 2020, 8:10 AM
State	Finished
Completed on	Wednesday, 7 October 2020, 9:13 AM
Time taken	1 hour 2 mins
Marks	58.58/80.00
Grade	73.23 out of 100.00

Question 1

Correct

Mark 1.00 out of 1.00

Berikut merupakan contoh overloading method yang diperbolehkan :

```
public void print ( int a)
public int print ( int a)
```

Select one:

- ☐ True
- ☒ False ✓

The correct answer is 'False'.

Question 2

Correct

Mark 1.00 out of 1.00

Berikut merupakan pernyataan yang salah tentang teori dasar class dan objek, yaitu :

Select one:

- ☐ a. Isi dari class terbagi menjadi dua bagian besar yaitu deklarasi variabel dan deklarasi method.
- ☐ b. Setiap objek memiliki property-nya masing-masing dimana perubahan property pada suatu instance tidak mempengaruhi nilai dari property yang sama yang terdapat pada instance lainnya
- ☐ c. Dengan mendeklarasikan suatu class, kita mendeklarasikan suatu tipe data baru yang dapat dibuat instancinya.
- ☒ d. Tipe data class bukan merupakan tipe data referensi, karena variabel dengan tipe data ini tidak memegang referensi dari objek yang sesungguhnya ✓
- ☐ e. Variabel yang didefinisikan di dalam class dikenal juga dengan nama property

The correct answer is: Tipe data class bukan merupakan tipe data referensi, karena variabel dengan tipe data ini tidak memegang referensi dari objek yang sesungguhnya

Question 3

Incorrect

Mark 0.00 out of 1.00

Pernyataan adalah Perintah yang menyebabkan sesuatu terjadi dan merepresentasikan suatu aksi tunggal dalam program Java. Didalam pernyataan pada java, terdapat nilai balik atau return value. Manakah yang salah dari point dibawah ini tentang return value

Select one:

- ☐ a. Method dengan nilai balik biasanya menggunakan keyword void
- ☐ b. Nilai balik bisa berupa bilangan, boolean, atau objek
- ☒ c. Contoh nilai balik : hasilBagi = a / b ✗
- ☐ d. Pernyataan dapat menghasilkan suatu nilai. Nilai yang dihasilkan oleh pernyataan ini yang disebut dengan nilai balik atau return value
- ☐ e. Nilai balik bisa berupa karakter atau string

The correct answer is: Method dengan nilai balik biasanya menggunakan keyword void

Question **4**

Incorrect

Mark 0.00 out of 2.00

Lengkapi kode dibawah :

```
public class Employee {

    String name;
    int age;
    String designation;
    double salary;

    // This is the public constructor of the class Employee
    Employee(nar ✖ {
        this.name = name;

    }

    public static void main(String args[]) {
        /* Create one objects using constructor */
        Employee empOne = new Employee("James Smith");

    }
}
```

The correct answer is: public Employee(String name)

Question **5**

Correct

Mark 2.00 out of 2.00

```
public class Car {

    public void fullThrottle() {
        System.out.println("The car is going as fast as it can!");
    }

    public void speed(int maxSpeed) {
        System.out.println("Wow, max speed is (km/hour) : " + maxSpeed);
    }

    public static void main(String[] args) {
        Car myCar = new Car();    // Create a myCar object
        myCar.speed(250);

    }
}
```

Output dari kode diatas jika dijalankan yaitu : Wow, max speed is (km/hour' ✔

The correct answer is: Wow, max speed is (km/hour) : 250

Question **6**
Correct
Mark 1.00 out of 1.00

Istilah untuk melindungi data dari usaha modifikasi, perusakan dan penggandaan data oleh pihak yang tidak berwenang adalah

Select one:

- ☐ a. Pewarisan
- ☐ b. Polymorphisme
- ☒ c. Encapsulation ✓
- ☐ d. Inheritance
- ☐ e. Constructor

The correct answer is: Encapsulation

Question **7**
Correct
Mark 2.00 out of 2.00

```
public class MyClass {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = 3;  
        System.out.println(x > y);  
    }  
}
```

Apakah output dari kode program diatas : ✓

The correct answer is: true

Question **8**
Incorrect
Mark 0.00 out of 2.00

Call **myMethod** on the object.

```
public class MyClass {  
    public [ myMethod( ✖ {  
        System.out.println("Hello World");  
    }  
  
    public static void main(String[] args) {  
        MyClass myObjek = new MyClass();  
        myObjek.myMethod();  
    }  
}
```

The correct answer is: void myMethod()

Question **9**
Correct
Mark 1.00 out of 1.00

Berikut merupakan contoh method overloading yang tidak tepat :

```
public void print ( int a, String s)  
public void print ( String s, int a )
```

Select one:

- ☐ True
- ☒ False ✓

The correct answer is 'False'.

Question **10**

Correct

Mark 1.00 out of 1.00

The most basic control flow statement supported by the Java programming language is the
statement

if-then



Your answer is correct.

The correct answer is:

The most basic control flow statement supported by the Java programming language is the [if-then] statement

Question **11**

Incorrect

Mark 0.00 out of 2.00

```
/*
 * The Circle class models a circle with a radius and color.
 */
public class Circle { // Save as "Circle.java"
    // private instance variable, not accessible from outside this class
    private double radius;
    private String color;

    // The default constructor with no argument.
    // It sets the radius and color to their default value.
    public Circle() {
        radius = 1.0;
        color = "red";
    }

    // 2nd constructor with given radius, but color default. The type of parameter is primitive data type
    public Circle {
        this.radius = radius;
        color = "red";
    }

    // A public method for retrieving the radius
    public double getRadius() {
        return radius;
    }

    // A public method for computing the area of circle
    public double getArea() {
        return radius*radius*Math.PI;
    }
}
```

The correct answer is: public Circle(double radius)

Question **12**
Partially correct
Mark 3.75 out of 5.00

Create and call a **class constructor** of MyClassB

Follow the comments to insert the missing parts of the code below:

```
public class MyClassB {  
    int a  ✓ ; // Create a class attribute  
  
    // Create a class constructor  
    public  MyClassB ✓ () {  
        a =  200 ✓ ; // Set the initial value for the class attribute to 200  
    }  
  
    public static void main(String[] args) {  
        // Create an object of class MyClassB (This will call the constructor)  
        MyClassB  MyObjek ✗ = new MyClassB();  
    }  
}
```

Your answer is partially correct.

You have correctly selected 3.

The correct answer is:

Create and call a **class constructor** of MyClassB

Follow the comments to insert the missing parts of the code below:

```
public class MyClassB {  
    int [a]; // Create a class attribute  
  
    // Create a class constructor  
    public [MyClassB]() {  
        a = [200]; // Set the initial value for the class attribute to 200  
    }  
  
    public static void main(String[] args) {  
        // Create an object of class MyClassB (This will call the constructor)  
        MyClassB [myClassBe] = new MyClassB();  
    }  
}
```

Question **13**

Partially correct

Mark 2.00 out of 3.00

Lengkapi kode dibawah dengan men-drag jawaban yang tepat ke dalam kode yang masih kosong :

```
class Bicycle
{
    // the Bicycle class has two fields
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int gear, int speed)
    {
        this.gear = gear;
        this.speed = speed;
    }

    // the Bicycle class has three methods
    public void applyBrake(int decrement)
    {
        speed -= decrement;
    }

    public void speedUp(int increment)
    {
        speed += increment;
    }

    // toString() method to print info of Bicycle
    public String toString()
    {
        return("No of gears are "+gear
            +"\n"
            + "speed of bicycle is "+speed);
    }
}

// derived class
class MountainBike  ☒ Bicycle
{

    // the MountainBike subclass adds one more field
    public int seatHeight;

    // the MountainBike subclass has one constructor
    public MountainBike(int gear,int speed,
        int startHeight)
    {
        // invoking base-class(Bicycle) constructor
         ☒ ;
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one more method
    public void setHeight(int newValue)
    {
        seatHeight = newValue;
    }

    // overriding toString() method
    // of Bicycle to print more info
    @Override
    public String toString()
    {
        return (  ☒ .toString()+
            "\nseat height is "+seatHeight);
    }
}
```



```
}

// driver class
public class Test
{
    public static void main(String args[])
    {

        MountainBike mb = new MountainBike(3, 100, 25);
        System.out.println(mb.toString());

    }
}



super()



super



Extends



super(speed)


```

Your answer is partially correct.

You have correctly selected 2.

The correct answer is:

Lengkapi kode dibawah dengan men-drag jawaban yang tepat ke dalam kode yang masih kosong :

```
class Bicycle
{
    // the Bicycle class has two fields
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int gear, int speed)
    {
        this.gear = gear;
        this.speed = speed;
    }

    // the Bicycle class has three methods
    public void applyBrake(int decrement)
    {
        speed -= decrement;
    }

    public void speedUp(int increment)
    {
        speed += increment;
    }

    // toString() method to print info of Bicycle
    public String toString()
    {
        return("No of gears are "+gear
            +"\n"
            + "speed of bicycle is "+speed);
    }
}

// derived class
class MountainBike [extends] Bicycle
{

    // the MountainBike subclass adds one more field
    public int seatHeight;

    // the MountainBike subclass has one constructor
    public MountainBike(int gear,int speed,
```

```

        int startHeight)
    {
        // invoking base-class(Bicycle) constructor
        [super(gear, speed)];
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one more method
    public void setHeight(int newValue)
    {
        seatHeight = newValue;
    }

    // overriding toString() method
    // of Bicycle to print more info
    @Override
    public String toString()
    {
        return ([super].toString()+
                "\nseat height is "+seatHeight);
    }
}

// driver class
public class Test
{
    public static void main(String args[])
    {

        MountainBike mb = new MountainBike(3, 100, 25);
        System.out.println(mb.toString());

    }
}

```

Question **14**

Correct

Mark 1.00 out of 1.00

Sebutkan tiga prinsip utama dalam pemrograman berorientasi objek

Select one:

- ☐ a. Encapsulation, polymorphism, extend
- ☐ b. Polymorphism, interface, encapsulation
- ☐ c. Polymorphism, inheritance, class
- ☐ d. Public, protected, private
- ☒ e. Inheritance, polymorphism, encapsulation ✓

The correct answer is: Inheritance, polymorphism, encapsulation

Question **15**

Incorrect

Mark 0.00 out of 1.00

Berikut merupakan keyword atau kata kunci pada Bahasa java yaitu, kecuali

Select one:

- ☒ a. native ✗
- ☐ b. implement
- ☐ c. super
- ☐ d. interface
- ☐ e. new

The correct answer is: implement

Question **16**

Correct

Mark 1.00 out of 1.00

The do-while statement is similar to the while statement, but evaluates its expression at the  of the loop

Your answer is correct.

The correct answer is:

The do-while statement is similar to the while statement, but evaluates its expression at the [bottom] of the loop

Question **17**

Incorrect

Mark 0.00 out of 2.00

```
public class Puppy {
    int puppyAge;

    public Puppy(String name) {
        // This constructor has one parameter, name.
        System.out.println("Name chosen is :" + name );
    }

    public void setAge(int age) {
        puppyAge = age;
    }

    public int getAge() {
        System.out.println("Puppy's age is :" + puppyAge);
        return puppyAge;
    }

    public static void main(String []args) {
        /* Object creation */
        Puppy myPuppy = new Puppy("tommy");

        /* Call class method to set puppy's age */
        myPuppy.setAge(2);

        /* Call another class method to get puppy's age */
        myPuppy.getAge();

        /* You can access instance variable as follows as well */
        System.out.println("Variable Value :" + myPuppy.puppyAge);
    }
}
```

a mutator method name is (write complete method name with the modifier and parameter) : 

The correct answer is: public void setAge(int age)

Question **18**

Correct

Mark 1.00 out of 1.00

```
// filename Main.java

class Grandparent {

    public void Print() {

        System.out.println("Grandparent's Print()");

    }

}

class Parent extends Grandparent {

    public void Print() {

        System.out.println("Parent's Print()");

    }

}

class Child extends Parent {

    public void Print() {

        super.super.Print();

        System.out.println("Child's Print()");

    }

}

public class Main {

    public static void main(String[] args) {

        Child c = new Child();

        c.Print();

    }

}
```

Output dari program diatas yaitu : Compiler Error in super.super.Print()



Your answer is correct.

The correct answer is:

```
// filename Main.java

class Grandparent {

    public void Print() {

        System.out.println("Grandparent's Print()");

    }

}

class Parent extends Grandparent {

    public void Print() {

        System.out.println("Parent's Print()");

    }

}

class Child extends Parent {

    public void Print() {

        super.super.Print();

        System.out.println("Child's Print()");

    }

}

public class Main {
```

```
public static void main(String[] args) {  
    Child c = new Child();  
    c.Print();  
}  
}
```

Output dari program diatas yaitu : [Compiler Error in super.super.Print()]

Question **19**

Correct

Mark 2.00 out of
2.00

A ✓ is a user defined blueprint or prototype from which ✓ are created. It represents the set of properties or methods that are common to all objects of one type.

Your answer is correct.

The correct answer is:

A [class] is a user defined blueprint or prototype from which [object] are created. It represents the set of properties or methods that are common to all objects of one type.

Question **20**

Correct

Mark 1.00 out of 1.00

```
class Base {

    public static void show() {

        System.out.println("Base::show() called");

    }

}

class Derived extends Base {

    public static void show() {

        System.out.println("Derived::show() called");

    }

}

class Main {

    public static void main(String[] args) {

        Base b = new Derived();

        b.show();

    }

}
```

Output dari kelas main diatas jika dijalankan, yaitu : ✓

Your answer is correct.

The correct answer is:

```
class Base {

    public static void show() {

        System.out.println("Base::show() called");

    }

}

class Derived extends Base {

    public static void show() {

        System.out.println("Derived::show() called");

    }

}

class Main {

    public static void main(String[] args) {

        Base b = new Derived();

        b.show();

    }

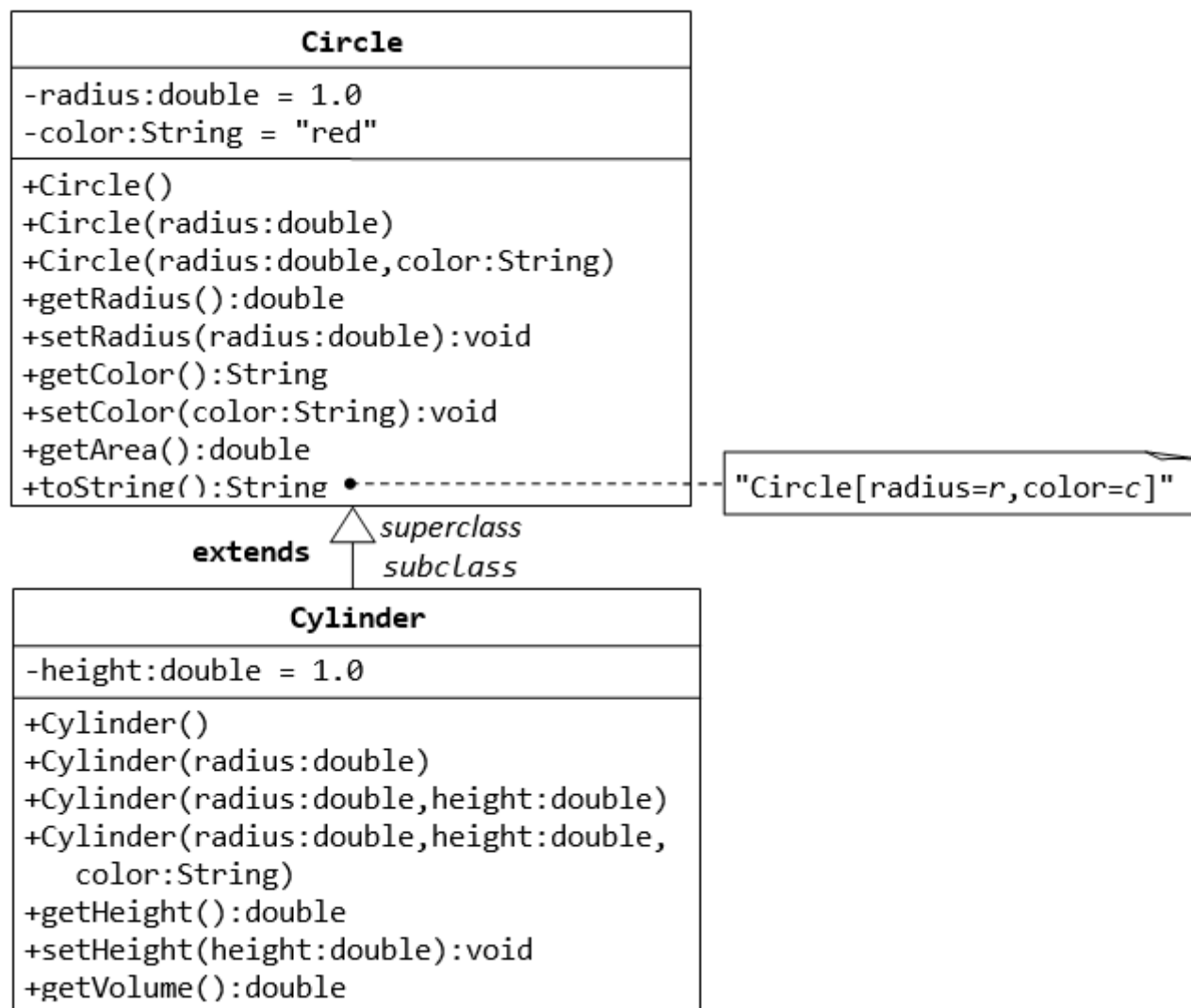
}
```

Output dari kelas main diatas jika dijalankan, yaitu : [Base::show() called]

Question **21**

Correct

Mark 10.00 out of 10.00



Berdasarkan diagram kelas diatas, lengkapi kode dibawah dengan men-drag jawaban yang tepat ke dalam kode program berikut :

```

public class Cylinder extends Circle { // Save as
"Cylinder.java"
    private double height ; // private variable

    // Constructor with default color, radius and height
    pub super()
        height = 1.0 ; // call superclass no-arg constructor Circle()
    }

    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        this call superclass no-arg constructor Circle()
        .height = height;
    }

    // Constructor with default color, but given radius, height
    pub super(radius) radius, double height) {
        call superclass constructor Circle(r)
        this.height = height;
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public dou getArea()
        return *height;
    }
}
    
```

Cylinder()

```

public class TestCylinder { // save as "TestCylinder.java"
    public static void main (String[] args) {
        // Declare and allocate a new instance of cylinder
        //   with default color, radius, and height
        Cylinder c1 = new  ✓ ;
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        //   specifying height, with default color and radius
        Cylinder c2 = new  ✓ ;
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        //   specifying radius and height, with default color
        Cylinder c3 = new  ✓ ;
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + 
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());
    }
}

```

This

10

Cylinder(2, 10.0)

Extends

super(height)

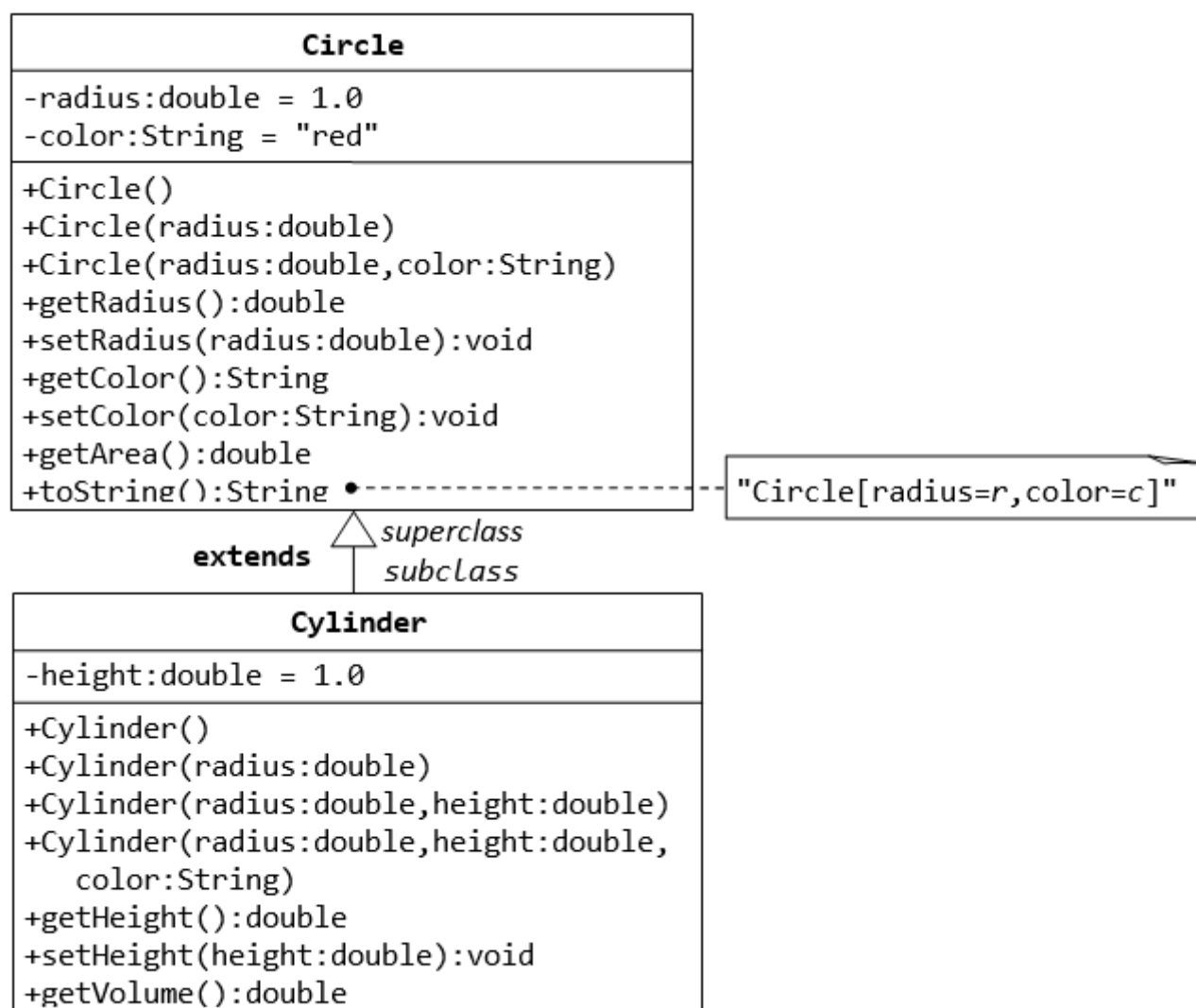
super

height = 1

Cylinder(20)

Your answer is correct.

The correct answer is:



Berdasarkan diagram kelas diatas, lengkapi kode dibawah dengan men-drag jawaban yang tepat ke dalam kode program berikut :


```

public class [Cylinder] [extends] [Circle] { // Save as "Cylinder.java"
    private double [height]; // private variable

    // Constructor with default color, radius and height
    public Cylinder() {
        [super()]; // call superclass no-arg constructor Circle()
        [height = 1.0];
    }
    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        super(); // call superclass no-arg constructor Circle()
        [this].height = height;
    }
    // Constructor with default color, but given radius, height
    public Cylinder(double radius, double height) {
        [super(radius)]; // call superclass constructor Circle(r)
        this.height = height;
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return [getArea()]*height;
    }
}

```

```

public class TestCylinder { // save as "TestCylinder.java"
    public static void main (String[] args) {
        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new [Cylinder()] ;
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new [Cylinder(10.0)];
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new [Cylinder(2.0, 10.0)];
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + c3.getHeight()
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());
    }
}

```

Question **22**

Incorrect

Mark 0.00 out of 1.00

String



methods are the methods in Java that can be called without creating an object of class. They are referenced by the **class name itself** or reference to the Object of that class.

The correct answer is: static

Question **23**

Correct

Mark 8.00 out of 8.00

```
class Drone {

    int energi;

    int ketinggian;

    int kecepatan;

    String merek;

    void terbang(){
        energi--;
        if(energi > 10){
            // terbang berarti nilai ketinggian bertambah
            ketinggian++;
            System.out.println("Drone terbang...");
        } else {
            System.out.println("Energi lemah: Drone nggak bisa terbang");
        }
    }

    void matikanMesin(){
        if( ketinggian > 0 ){
            System.out.println("Mesin tidak bisa dimatikan karena sedang terbang");
        } else {
            System.out.println("Mesin dimatikan...");
        }
    }

    void turun(){
        // ketinggian berkurang, karena turun
        ketinggian--;
        energi--;
        System.out.println("Drone turun");
    }

    void belok(){
        energi--;
        System.out.println("Drone belok");
    }

    void maju(){
        energi--;
        System.out.println("Drone maju ke depan");
        kecepatan++;
    }

    void mundur(){
        energi--;
        System.out.println("Drone mundur");
        kecepatan++;
    }
}
```


Your answer is correct.

The correct answer is:

```
[class] [Drone] {
    int energi;
    int ketinggian;
    int kecepatan;
    [String] merek;

    void terbang(){
        energi--;
        if(energi > 10){
            // terbang berarti nilai ketinggian bertambah
            ketinggian++;
            System.out.println("Drone terbang...");
        } else {
            System.out.println("Energi lemah: Drone nggak bisa terbang");
        }
    }

    void matikanMesin(){
        if([ketinggian > 0]){
            System.out.println("Mesin tidak bisa dimatikan karena sedang terbang");
        } else {
            System.out.println("Mesin dimatikan...");
        }
    }

    void turun(){
        // ketinggian berkurang, karena turun
        ketinggian--;
        energi--;
        System.out.println("Drone turun");
    }

    void belok(){
        energi--;
        System.out.println("Drone belok");
    }

    void maju(){
        energi--;
        System.out.println("Drone maju ke depan");
        kecepatan++;
    }

    void mundur(){
        energi--;
        System.out.println("Drone mundur");
        kecepatan++;
    }
}
```

Question **24**

Correct

Mark 2.00 out of 2.00

```
public class MyClass {  
    public static void main(String[] args) {  
        int x = 15;  
        int y = 2;  
        System.out.println(x / y);  
    }  
}
```

Apakah output dari kode program diatas :

7



The correct answer is: 7

Question **25**

Incorrect

Mark 0.00 out of 1.00

Berikut adalah penamaan class pada java yang diperbolehkan, kecuali

Select one:

- ☒ a. s4tu ✖
- ☐ b. O_3ne
- ☐ c. S13h
- ☐ d. 3_One
- ☐ e. B3_Ta

The correct answer is: 3_One

Question **26**

Correct

Mark 1.00 out of 1.00

Istilah lain dari atribut yaitu kecuali

Select one:

- ☐ a. Data
- ☐ b. Variabel
- ☐ c. State
- ☐ d. Member
- ☒ e. Behavior ✔

The correct answer is: Behavior

Question **27**

Correct

Mark 1.00 out of 1.00

Di dalam pemrograman berorientasi objek dilakukan seleksi entitas dengan mempertimbangkan relevansi dengan permasalahan yang ingin dibuatkan aplikasi PBOnya. Tahapan seleksi entitas ini berada pada prinsip utama PBO yaitu :

Select one:

- ☒ a. Abstraksi ✔
- ☐ b. Pewarisan
- ☐ c. Static class
- ☐ d. Enkapsulasi
- ☐ e. Pengaturan akses modifier

The correct answer is: Abstraksi

Question **28**
Correct
Mark 2.00 out of 2.00

```
public class MyClass {  
  
    public static void main(String[] args) {  
  
        int x = 5;  
  
        System.out.println(x > 3 && x < 5 );  
  
    }  
}
```

Apakah output dari kode program diatas : ✓

The correct answer is: false

Question **29**
Partially correct
Mark 2.00 out of 4.00

There are access modifiers for attributes, methods and constructors at java :

- | | | |
|--|---|--|
| <input type="text" value="protected"/> | ✗ | The code is only accessible in the same package. This is used when you don't specify a modifier. |
| <input type="text" value="default"/> | ✗ | The code is accessible in the same package and subclasses. |
| <input type="text" value="public"/> | ✓ | The code is accessible for all classes |
| <input type="text" value="private"/> | ✓ | The code is only accessible within the declared class |

Your answer is partially correct.

You have correctly selected 2.

The correct answer is:

There are access modifiers for attributes, methods and constructors at java :

- [default] The code is only accessible in the same package. This is used when you don't specify a modifier.
- [protected] The code is accessible in the same package and subclasses.
- [public] The code is accessible for all classes
- [private] The code is only accessible within the declared class

Question **30**
Correct
Mark 1.00 out of 1.00

(Completing the code) Convert the following **double** type (myDouble) to an **int** type :

```
double myDouble = 9.78;  
int myInt =  ✓ myDouble;
```

The correct answer is: (int)

Question **31**
Correct
Mark 2.00 out of 2.00

The Java codes are first compiled into byte code (machine independent code). Then the byte code is run on JVM regardless of the underlying architecture. JVM stands for ✓

The correct answer is: Java Virtual Machine

Question **32**

Partially correct

Mark 2.00 out of 4.00

The following table summarises the above said points regarding the accessibility of entities. We haven't looked into what a subclass is and the accessibility in a subclass. We shall see about it when we deal with inheritance. Drag the correct access specifier to the table bellow :

		class	subclass	package	outside
private	✓	allowed	not allowed	not allowed	not allowed
default	✗	allowed	allowed	allowed	not allowed
public	✓	allowed	allowed	allowed	allowed
protected	✗	allowed	not allowed	allowed	not allowed

specified

access specifier

Your answer is partially correct.
You have correctly selected 2.
The correct answer is:

The following table summarises the above said points regarding the accessibility of entities. We haven't looked into what a subclass is and the accessibility in a subclass. We shall see about it when we deal with inheritance. Drag the correct access specifier to the table bellow :

	class	subclass	package	outside
[private]	allowed	not allowed	not allowed	not allowed
[protected]	allowed	allowed	allowed	not allowed
[public]	allowed	allowed	allowed	allowed
[default]	allowed	not allowed	allowed	not allowed

Question **33**

Correct

Mark 1.00 out of 1.00

In order to obtain the Java Array Length, complete the **code** below:

```
1  /**
2   * An Example to get the Array Length is Java
3   */
4   public class ArrayLengthJava {
5       public static void main(String[] args) {
6           String[] myArray = { "I", "Love", "Lombok" };
7           int arrayLength = myArray. length; ✓ //array length attribute
8           System.out.println("The length of the array is: " + arrayLength);
9       }
10  }
```

Output

The length of the array is: 3

The correct answer is: length;

Question **34**
Partially correct
Mark 5.83 out of 7.00

Lengkapi kode dibawah dengan men-drag jawaban yang tepat :

```
public class Student {  
  
    private int rollNumber;  
    private String  ✓ ;  
    private int marks1;  
    private int marks2;  
    private int marks3;  
  
    public void  ✓ (String s) {  
        name = s;  
    }  
  
    public String  ✓ () {  
        return name;  
    }  
  
    public void  ✓ (int r) {  
        if (r > 0) {  
            rollNumber = r;  
        } else {  
            rollNumber = 1;  
        }  
    }  
  
    public int  ✓ () {  
        return rollNumber;  
    }  
  
    public void  ✓ (int m) {  
        if (m >= 0 && m <= 100) {  
            marks1 = m;  
        } else {  
            marks1 = 0;  
        }  
    }  
  
    public int  ✓ () {  
        return marks1;  
    }  
  
    public void  ✓ (int m) {  
        if ((m >= 0) && (m <= 100)) {  
            marks2 = m;  
        } else {  
            marks2 = 0;  
        }  
    }  
  
    public int  ✓ () {  
        return marks2;  
    }  
  
    public void  ✗ (int m) {  
        if ((m >= 0) && m <= 100) {  
            marks3 = m;  
        } else {  
            marks3 = 0;  
        }  
    }  
  
    public int  ✗ () {  
        return marks3;  
    }  
}
```

```

    }

    public double getAverage() {
        return (marks1+ marks2 + marks3) / 3.0;
    }

    public void printDetails() {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Marks in first subject: " + marks1);
        System.out.println("Marks in second subject: " + marks2);
        System.out.println("Marks in second subject: " + marks3);
        System.out.println("Average: " + getAverage());
    }
}

```

The following is the StudentTest class.

```
import java.util.Scanner;
```

```
public class StudentTest {
```

```

    public static void main(String[] args) {
        Student student = new Student();
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter ");
        String name = scanner.
        student.setName(name);
        System.out.print("Enter roll number: ");
        int rollNumber = scanner.nextInt();
        student.setRollMumber(rollNumber);
        System.out.print("Enter marks1: ");
        int marks1 = scanner.nextInt();
        student.setMarks1(marks1);
        System.out.print("Enter marks2: ");
        int marks2 = scanner.nextInt();
        student.setMarks1(marks2);
        System.out.print("Enter marks3: ");
        int marks3 = scanner.nextInt();
        student.setMarks1(marks3);
        System.out.println();
        System.out.println("Printing details using printDetails() method: =");
        student.printDetails();
        System.out.println();
        System.out.println("Printing details without using printDetails() method:");
        System.out.println("Name: " + student.getName());
        System.out.println("Roll Number: " + student.getRollNumber());
        System.out.println("Marks1: " + student.getMarks1());
        System.out.println("Marks2: " + student.getMarks2());
        System.out.println("Marks3: " + student.getMarks3());
        System.out.println("Average: " + student.getAverage());
    }
}

```

nextLine()



nextInt()

nextString()

Your answer is partially correct.

You have correctly selected 10.

The correct answer is:

Lengkapi kode dibawah dengan men-drag jawaban yang tepat :


```
public class Student {

    private int rollNumber;
    private String [name];
    private int marks1;
    private int marks2;
    private int marks3;

    public void [setName](String s) {
        name = s;
    }

    public String [getName]() {
        return name;
    }

    public void [setRollNumber](int r) {
        if (r > 0) {
            rollNumber = r;
        } else {
            rollNumber = 1;
        }
    }

    public int [getRollNumber]() {
        return rollNumber;
    }

    public void [setMarks1](int m) {
        if (m >= 0 && m <= 100) {
            marks1 = m;
        } else {
            marks1 = 0;
        }
    }

    public int [getMarks1]() {
        return marks1;
    }

    public void [setMarks2](int m) {
        if ((m >= 0) && (m <= 100)) {
            marks2 = m;
        } else {
            marks2 = 0;
        }
    }

    public int [getMarks2]() {
        return marks2;
    }

    public void [setMarks3](int m) {
        if ((m >= 0) && m <= 100) {
            marks3 = m;
        } else {
            marks3 = 0;
        }
    }

    public int [getMarks3]() {
        return marks3;
    }

    public double getAverage() {
        return (marks1+ marks2 + marks3) / 3.0;
    }
}
```

```
public void printDetails() {  
    System.out.println("Roll Number: " + rollNumber);  
    System.out.println("Name: " + name);  
    System.out.println("Marks in first subject: " + marks1);  
    System.out.println("Marks in second subject: " + marks2);  
    System.out.println("Marks in second subject: " + marks3);  
    System.out.println("Average: " + getAverage());  
}  
}
```

The following is the StudentTest class.

```
import java.util.Scanner;  
  
public class StudentTest {  
  
    public static void main(String[] args) {  
        Student student = new Student();  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter Name: ");  
        String name = scanner.nextLine();  
        student.setName(name);  
        System.out.print("Enter roll number: ");  
        int rollNumber = scanner.nextInt();  
        student.setRollMumber(rollNumber);  
        System.out.print("Enter marks1: ");  
        int marks1 = scanner.nextInt();  
        student.setMarks1(marks1);  
        System.out.print("Enter marks2: ");  
        int marks2 = scanner.nextInt();  
        student.setMarks1(marks2);  
        System.out.print("Enter marks3: ");  
        int marks3 = scanner.nextInt();  
        student.setMarks1(marks3);  
        System.out.println();  
        System.out.println("Printing details using printDetails() method: =");  
        student.printDetails();  
        System.out.println();  
        System.out.println("Printing details without using printDetails() method:");  
        System.out.println("Name: " + student.getName());  
        System.out.println("Roll Number: " + student.getRollNumber());  
        System.out.println("Marks1: " + student.getMarks1());  
        System.out.println("Marks2: " + student.getMarks2());  
        System.out.println("Marks3: " + student.getMarks3());  
        System.out.println("Average: " + student.getAverage());  
    }  
}
```


Question **35**
Incorrect
Mark 0.00 out of 2.00

Lengkapi kode dibawah agar kode dapat dirunning tanpa adanya error :

```
public class MyClass {  
  
    static void MyClass()  {  
        System.out.println("Static methods can be called without creating objects");  
    }  
  
    public void myPublicMethod() {  
        System.out.println("Public methods must be called by creating objects");  
    }  
  
    public static void main(String[] args) {  
        myMethodPagi();  
        MyClass myObj = new MyClass();  
        myObj.myPublicMethod();  
    }  
}
```

The correct answer is: myMethodPagi()